

Cours + TD + BE de méthodes numériques

Intervenants pour la filière PNS :

Jean-Christophe Toussaint

Quentin Rafhay – Yves Duterrail

60 Heures (15 séances de 4 heures)

Minatec sud salle info spécialisée

Présence obligatoire

Pourquoi un cours de méthodes numériques ?

Pour faire de la modélisation de phénomènes physiques

Cours + TD + BE de méthodes numériques

Intervenants pour la filière GEN :

Jean-Christophe Toussaint

Marc Fivel – Adrien Bidaud

60 Heures (15 séances de 4 heures)

Minatec sud salle info spécialisée

Présence obligatoire

Pourquoi un cours de méthodes numériques ?

Pour faire de la modélisation de phénomènes physiques



La modélisation : Mais à quoi ça sert ?

Modélisation (*dictionnaire Robert*)

Mise en équation d'un phénomène complexe permettant d'en prévoir les évolutions

Simulation

Processus qui permet de reproduire numériquement sur ordinateur un phénomène décrit par un modèle mathématique.

Evolutions : spatiales et temporelles

Equations : lois de conservation (flux thermique, flux de matière, équations de Maxwell ...)

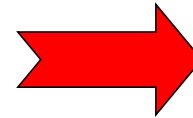
Equations aux Dérivées Partielles (EDP) : $\partial / \partial x$, $\partial^2 / \partial x^2$, $\partial / \partial t$, $\partial^2 / \partial t^2$

La modélisation de phénomènes physiques

La **Modélisation** : Mais à quoi ça sert ?

Les intérêts de simuler le fonctionnement de dispositifs réels sont :

- pour **mieux les concevoir**
- pour **optimiser le fonctionnement**



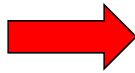
Cadre de la D.A.O.



Cadre des Math. Appliquées
à la Physique, à la Biologie, à
la Finance

Etapes de la Simulation Numérique

Phénomène Physique
(Expériences – Mesures)



Définition des variables d'état (par ex. T)



Approximation : dépendance des propriétés physiques en fonction des variables d'état (par ex. $k_{th}=f(T)$)



Approximation de la réponse du système : (par ex.)

$$\mathbf{j} = -k_{th} \text{ grad } T$$



Lois de conservation (par ex. du flux de chaleur)

⇒ Modèle Physico-Mathématique :

Équations différentielles EDO (temps) :

variable d'état $\theta(t)$ $\frac{d^2\theta}{dt^2} + \frac{g}{l} \sin \theta = 0$

ou

Equations aux dérivées partielles EDP (temps et espace) :

variable d'état $T(x, t)$ $-\frac{\partial}{\partial x} \left(k_{th}(T) \frac{\partial T}{\partial x} \right) + \rho c_p \frac{\partial T}{\partial t} = Q$

Etapes de la Simulation Numérique



Analyse mathématique des EDO et EDP :
 $\exists$? Une solution analytique sur des géométries simples ?



Analyse numérique des EDO et EDP :
discrétisation en temps-espace $\Rightarrow$ écriture de schémas numériques
Estimation d'erreurs – Algorithme de résolution



Programmation sur ordinateur avec un langage de haut niveau:
Débogage, Validation avec des cas tests,
Visualisation et Post-traitement des résultats.
Confrontation avec des mesures sur systèmes réels
Optimisation,
Mise en exploitation, retour d'expériences.

Plan du cours de méthodes numériques

Généralités - Introduction à Matlab (2 séances)

Résolution numérique d'équations différentielles (2 séances)

Résolution d'équations aux dérivées partielles - 1

Méthode des différences finies (3 séances)

- Schéma explicite

- Schéma explicite + implicite

- Schéma implicite

Résolution des systèmes linéaires et non linéaires (1 séance)

Interpolation et intégration numérique (2 séances)

Résolution d'équations aux dérivées partielles -2

Méthode des éléments finis (5 séances)

- Approche de Galerkin sur EDP électrocinétique

- Approche de Galerkin sur EDP électrostatique

- Approche de Galerkin sur EDP Thermique

Introduction à Matlab

- Logiciel commercial développé par The MathWorks Inc.
- Calcul numérique sur des grandeurs vectorielles et matricielles, réelles ou complexes
- Clône de Matlab = octave
- Fonctionne sous Unix et Windows

Prise en main

- Dans une fenêtre terminal, taper matlab &

The image shows the MATLAB R2013a desktop environment. The main window has a ribbon-style toolbar with tabs for HOME, PLOTS, and APPS. Below the toolbar are several panes: Current Folder, Command Window, Workspace, and a History pane. A 'Help' window is open in the foreground, displaying a search bar and a list of toolboxes. Annotations with arrows point to specific elements:

- Invite de commande**: Points to the Command Window, which displays the MATLAB logo and a license notice.
- Variables déclarées**: Points to the Workspace pane, which shows a table of variables.
- Cliquer sur *help/MATLAB help* pour accéder à l'aide**: Points to the 'MATLAB' link in the Help window's list of toolboxes.
- Historique des commandes**: Points to the History pane, which shows a list of commands entered in the Command Window.

Introduction d'une matrice

--> A=[1 1 1; 2 4 8; 3 9 27] ↩

A =

1.	1.	1.
2.	4.	8.
3.	9.	27.

Définition de la Matrice :

- chaque composante séparée par un blanc.
- chaque ligne par un point-virgule

--> A'

1.	2.	3.
1.	4.	9.
1.	8.	27.

L'opérateur ' calcule la Matrice transposée conjuguée complexe



Comment calculer 2*A ?

Produit matrice-vecteur

--> `b = [1 0 0];`

Pas d'affichage du résultat si point-virgule

--> `A*b`

??? Error using ==> *

Inner matrix dimensions must agree.

--> `A*b'`

					1 0 0
	1.	1.	1.		
	2.	4.	8.		
	3.	9.	27.		= ?

	1.	
	2.	
	3.	

Opérateurs arithmétiques + - * /

Résolution de systèmes linéaires

--> $b = [1 \ 0 \ 0]'$;

transposée

--> $x = A \setminus b$

Résolution de $A*x = b$

$x =$

Vérifier que ça a marché !

$$\begin{bmatrix} 3.0 \\ -2.5 \\ 0.5 \end{bmatrix}$$

DANGER avec l'opérateur $\setminus$
 $2*A \setminus b$ est équivalent à $(2*A) \setminus b$

CONSEIL : mettre des parenthèses
en cas de doutes



Exercices

Définir un vecteur $V = \begin{bmatrix} 1 \\ -2 \\ 3 \end{bmatrix}$ une matrice $A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & -3 & 5 \\ 4 & -3 & 1 \end{bmatrix}$

- Afficher la 2ème composante de V
- Afficher l'élément de la matrice A situé
en 2ème ligne et 3ème colonne
- Afficher la 1ère colonne de A
- Afficher la 2ème ligne de A
- Calculer $A*V$
- Résoudre $A X = V$
- Afficher le résultat

```
>> V=[1 -2 3]' ou V=[1;-2;3]
```

```
V =
```

```
1  
-2  
3
```

```
>> V(1)
```

```
ans =
```

```
1
```

```
>> A=[1 2 3;2 -3 5;4 -3 1]
```

```
A =
```

```
1 2 3  
2 -3 5  
4 -3 1
```

```
>> A(2,3)
```

```
ans =
```

```
5
```

```
>> A(:,1)
```

```
ans =
```

```
1  
2  
4
```

```
>> A(2,:)
```

```
ans =
```

```
2 -3 5
```

```
>> W=A*V
```

```
W =
```

```
6  
23  
13
```

```
>> d = det(A)
```

```
d =
```

```
66
```

```
>> x = A\V
```

```
x =
```

```
1.3788  
0.6515  
-0.5606
```

```
>> x = inv(A)*V
```

```
x =
```

```
1.3788  
0.6515  
-0.5606
```

Construction de matrices et de vecteurs

Construire

- la matrice nulle de dimensions 5 x 10,
- la matrice unité de dimension 10,
- la matrice tri-diagonale T de taille 10 x 10, et le vecteur W suivants

$$T = \begin{bmatrix} 2 & -1 & 0 & \dots & 0 \\ -1 & 2 & -1 & \dots & 0 \\ 0 & -1 & \dots & \dots & 0 \\ 0 & \dots & \dots & \dots & -1 \\ 0 & \dots & \dots & -1 & 2 \end{bmatrix} \quad \text{et} \quad W = \begin{bmatrix} 1 \\ 2 \\ \vdots \\ 9 \\ 10 \end{bmatrix}$$

Calculer la dimension de la matrice T.

Extraire la ligne 5 et la colonne 10 de la matrice T.

Liste des fonctions MATLAB à voir : zeros, ones, diag, eye, size

Premier programme « Hello World ! »

- Edition du programme : File → New → M-file
- Sauvegarde : File → Save , taper hello.m
- Interprétation du code Matlab

```
C:\users\toussain\Enspg1\matlab_jct2004\hello.m
File Edit View Text Debug Breakpoints Web Window Help
1 % programme hello
2 disp('Hello World!');
```

```
C:\users\toussain\Enspg1\matlab_jct2004\hello.m
File Edit View Text Debug Breakpoints Web Window Help
Step F10
Step In F11
Step Out Shift+F11
Run F5
Go Until Cursor
Exit Debug Mode
1 % programme hello
2 disp('Hello World!');
```

ou en tapant le nom du programme
dans la fenêtre de commande

MATLAB
File Edit View Web Window Help
Current Directory: C:\users\toussain\Enspg1\matlab_jct2004

Workspace
Stack: Base

Name	Size	Bytes	Class
------	------	-------	-------

Command Window
>> hello
Hello World!
>>

Variables en Matlab

- Les variables en Matlab sont tous des tableaux, Matlab est sensible à la casse pour les noms de variables : `I` $\neq$ `i`
- Déclaration dynamique des variables lors de leur initialisation

Exemple :

```
t=0;
```

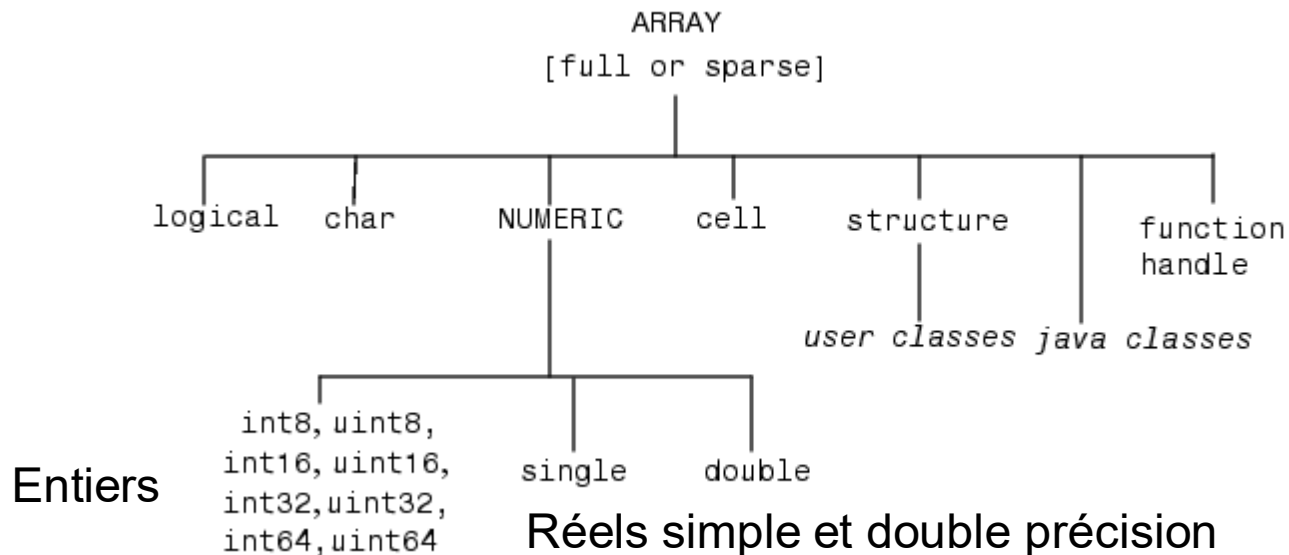
commentaire

```
% variable de type NUMERIC
```

```
s='Hello World';
```

```
% variable de type tableau de caractères
```

- Redéclaration possible d'une variable en changeant son type



```

C:\users\toussain\Enspg1\matlab_jct2004\declaration.m
File Edit View Text Debug Breakpoints Web Window Help
1 % programme declaration
2 - clear all
3 - disp(x) ;

```

Erroné !

```

C:\users\toussain\Enspg1\matlab_jct2004\declaration.m
File Edit View Text Debug Breakpoints Web Window Help
1 % programme declaration
2 - clear all
3 x=10;
4 - disp(x) ;

```

Ok !

Regarder l'aide sur zeros, ones et diag

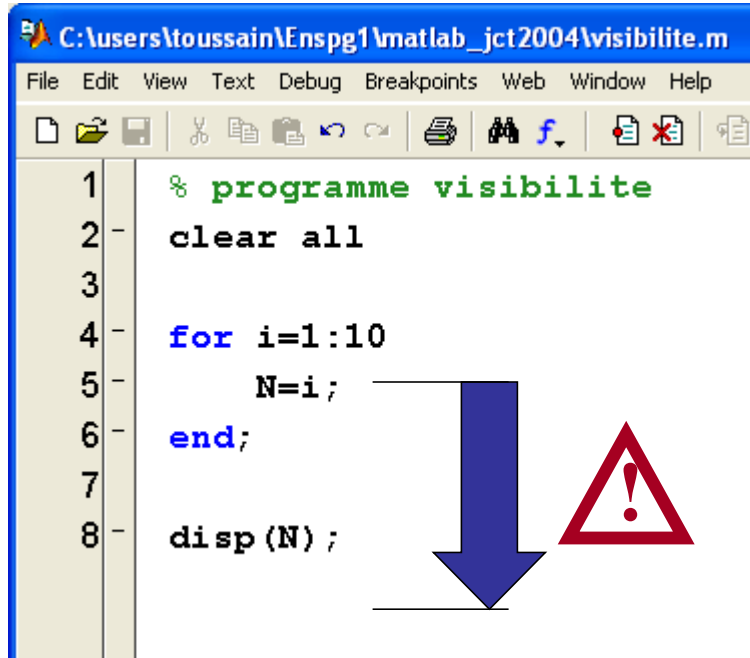
```

C:\users\toussain\Enspg1\matlab_jct2004\declaration2.m
File Edit View Text Debug Breakpoints Web Window Help
1 % programme declaration
2 - clear all
3 - N=3
4 - t=zeros(N,N)
5
6 - N=2
7 - t=ones(N,N)
8
9 - N=3
10 - t=diag(ones(N,1))

```

N =	N =	N =
3	2	3
t =	t =	t =
0 0 0	1 1	1 0 0
0 0 0	1 1	0 1 0
0 0 0		0 0 1

Visibilité des variables



The image shows a MATLAB editor window titled 'C:\users\Moussain\Enspg1\matlab_jct2004\visibilite.m'. The script contains the following code:

```
1 % programme visibilite
2 clear all
3
4 for i=1:10
5     N=i;
6 end;
7
8 disp(N);
```

A large blue arrow points from the line 'N=i;' to a red warning triangle icon, indicating a scope-related error.

Déclaration de la variable N
Portée : de la déclaration
Jusqu'à la fin du fichier

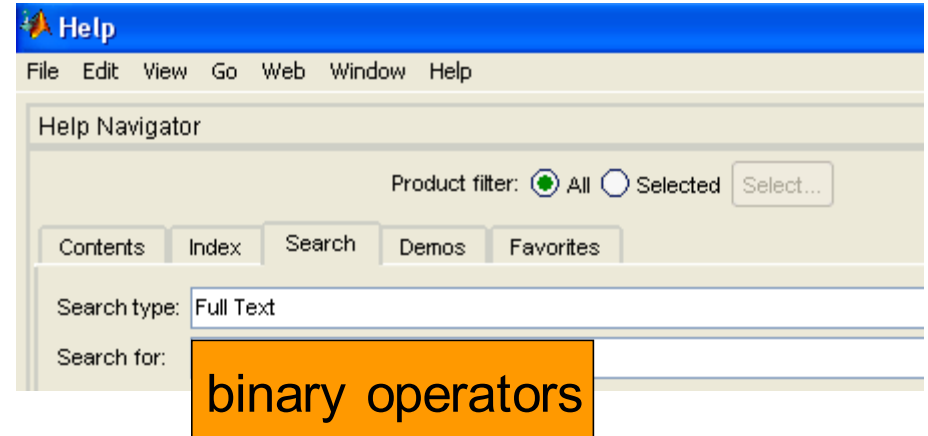
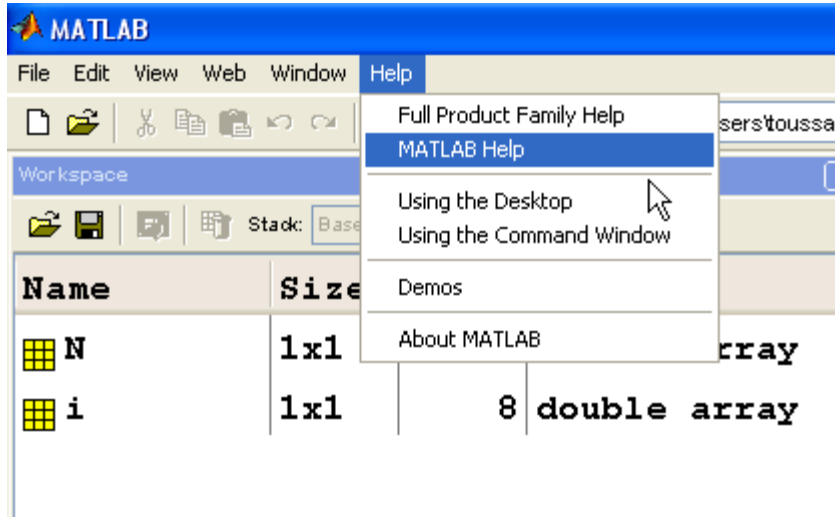
⇒ Pas de déclaration locale
À un bloc !!!

Danger : le remplacement de ':' par une virgule dans un for n'est pas signalé!

```
for i=1, 10
    i
end;
```

Opérateurs binaires

- A découvrir par vous-mêmes



- Sélectionner ensuite séquentiellement :

- arithmetic operators
- logical operators
- relational operators

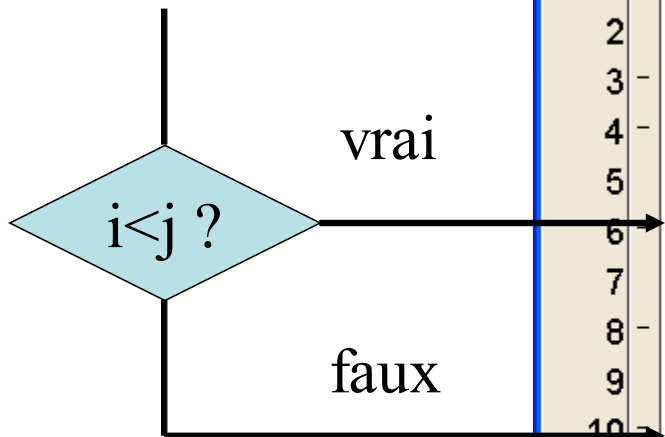
A && B
A || B

A < B
A > B
A <= B
A >= B
A == B
A ~= B

≠

Les structures de contrôle

La structure conditionnelle if



```
C:\users\toussain\Enspg1\matlab_jct2004\compare.m
File Edit View Text Debug Breakpoints Web Window Help
[Icons]
1 % programme compare
2
3 i=input('nombre i :');
4 j=input('nombre j :');
5
6 if (i<j)
7
8     disp('i<j');
9
10 else
11
12     disp('i>=j');
13
14 end;
15
16
```

Saisie des nombres
i & j

bloc d'instruction

Le branchement conditionnel

Cas où des instructions différentes doivent être exécutées selon la valeur d'une variable, l'écriture de **if** successifs est lourde.

```
C:\users\toussain\Enspg1\matlab_jct2004\choix.m
File Edit View Text Debug Breakpoints Web Window Help
[Icons]
1 % programme choix
2
3 choix=input('choix [1,3] :');
4
5 switch choix
6     case 1
7         disp('choix 1')
8     case 2
9         disp('choix 2')
10    case 3
11        disp('choix 3')
12    otherwise
13        disp('choix indetermine')
14 end
15
```

Pas d'instructions break dans un bloc **case** comme en C++!

```
C:\users\toussain\Enspg1\matlab_jct2004\choix2.m
File Edit View Text Debug Breakpoints Web Window Help
[Icons]
1 % programme choix2
2
3 choix=input('choix [1,3] :');
4
5 if (choix==1)
6     disp('choix 1')
7 else
8     if (choix==2)
9         disp('choix 2')
10    else
11        if (choix==3)
12            disp('choix 3');
13        else
14            disp('choix indetermine')
15        end
16    end
17 end
18
19
```

Les boucles

```
C:\users\toussain\Enspg1\matlab_jct2004\loop1.m
File Edit View Text Debug Breakpoints Web Window Help
% programme loop1
1
2
3 - for i=1:10
4 -     disp(i);
5 - end
6
```

```
1 % programme loop2
2
3 - i=1;
4 - while (i<=10)
5 -     disp(i);
6 -     i=i+1;
7 - end
```

```
C:\users\toussain\Enspg1\matlab_jct2004\loop3.m
File Edit View Text Debug Breakpoints Web Window Help
% programme loop3
1
2
3 - i=1;
4 - while (true)
5 -     disp(i);
6 -     i=i+1;
7 -     if (i>10)
8 -         break;
9 -     end
10 - end
11
```

```
C:\users\toussain\Enspg1\matlab_jct2004\sum1.m
File Edit View Text Debug Breakpoints Web Window Help
% programme sum1
1
2
3 - N=10;
4 - sum=0;
5 - for i=1:N
6 -     sum=sum+i;
7 - end
8
9 - disp(sum);
10 - disp(N*(N+1)/2);
11
```

Ce programme calcule-t-il

$$\sum_{i=0}^n i = \frac{n(n+1)}{2}$$

Break : rupture de boucle

Fonctions mathématiques

$|x|$

$\text{abs}(x)$

$\arctg x$

$\text{atan}(x)$

$\cos x$

$\cos(x)$

$\exp x$

$\exp(x)$

$\ln x$

$\log(x)$

$\log x$

$\log_{10}(x)$

$\sqrt{x}$

$\text{sqrt}(x)$

x^y

x^y

≠ C &
C++

```
C:\users\toussain\Enspg1\matlab_jct2004\complex1.m
File Edit View Text Debug Breakpoints Web Window Help
[Icons]
1 % programme complex1
2
3 x=input('argument en rad ');
4
5 disp(cos(x));
6 disp(sin(x));
7
8 disp(exp(1i*x));
```

$1i \equiv \text{sqrt}(-1.)$

```
1 % programme complex2
2
3 x=input('argument en rad ');
4
5 s= [num2str(cos(x)) ' + i ' ...
6     num2str(sin(x))];
```

... : continuation ligne suivante

num2str : conversion numerique → chaine

```
7 disp(s);
8
9 disp(exp(1i*x));
```


Ecriture d'une fonction (1)

- Ouvrir l'éditeur de texte : File → New → M-file

% resolution d'un systeme lineaire

% commentaire

function [x]=resolution(A, b)

Entrées séparées par une virgule

x = A\b;

Sorties séparées par une virgule si nécessaire

error(lastwarn);

end

Tout warning arrête le déroulement

- Sauver sous le nom "resolution.m" par exemple
- Dans matlab, sélectionner "resolution.m" avec le menu file → open
- Appeler la fonction de resolution depuis la fenêtre de commande

x = resolution(A, b)

linsolver.m

```
function linsolver
A=[1 2 3;2 -3 5;4 -3 1]
b=[1 -2 3] '

[x, ier]=resolution(A, b)
if (ier)
    error('Pb resolution');
end;

% resolution d'un systeme lineaire
function [x, res]=resolution(A, b)
try
    res = 0;
    x = A\b;
    error(lastwarn);
catch ME
    ME
    res = 1;
    x = 0;
end;
end
```

Ecriture d'une fonction (2)

linsolver est la fonction principale
appelant la routine de résolution

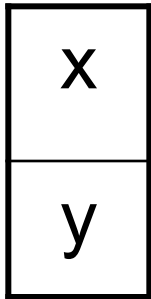
```
Branchement conditionnel
if (condition)
    bloc;
else
    bloc;
end
```

det(A) nul provoque un warning uniquement !
Transformation du warning en erreur

Récupération de l'erreur

Les structures

Point en cartésien 2D



```
M.x=input('abscisse du point : ');  
M.y=input('ordonnee du point : ');  
  
disp(M);
```

```
% programme struct1  
clear all
```

```
N=2;  
for i=1:N  
    s=['point ' num2str(i) ];  
    disp(s);  
    x=input('x : ');  
    y=input('y : ');  
    M(i).x=x;  
    M(i).y=y;  
end
```

```
for i=1:N  
    s=['point' num2str(i) ]  
    disp(M(i));  
end
```

Déclaration dynamique et
allocation dynamique
du tableau de structures
de points

Calcul de vecteurs propres par la méthode de la puissance itérée

- Soit A une matrice symétrique de dimensions n x n
- x0 un vecteur quelconque de dimension n
- On fixe un epsilon d'arrêt et un nombre maximal d'itérations

$$\hat{A} = \sum_i |u_i\rangle \lambda_i \langle u_i| \quad \text{avec}$$

$$\langle u_i | u_j \rangle = \delta_{ij}$$

$$\hat{A} |x_0\rangle = \sum_i |u_i\rangle \lambda_i \langle u_i | x_0 \rangle$$

$$\hat{A}^n |x_0\rangle = \sum_i |u_i\rangle \lambda_i^n \langle u_i | x_0 \rangle$$

$$\hat{A}^n |x_0\rangle = \lambda_1^n \left(\frac{\langle u_1 | x_0 \rangle}{\langle u_1 | u_1 \rangle} |u_1\rangle + \left(\frac{\lambda_2}{\lambda_1} \right)^n \langle u_2 | x_0 \rangle |u_2\rangle + \dots \right)$$

→ 0 quand n → ∞

Boucle sur les itérations

u ← A * x0

x ← u / norme(u)

si (norme(x-x0) < epsilon) alors
afficher norme(x-x0)

stop

sinon

x0 ← x

finsi

Fin boucle

u ← u / norme(u)



$$\Rightarrow \lambda_1 = \langle u_1 | \hat{A} | u_1 \rangle$$

lambda = u' * A * u;

```
% puissance iteree de mat A hermitique de dim nxn
% [lambda, u]=puissance(A, x0)
% x0 vecteur colonne de dim n
```

```
function [lambda, u]=puissance(A, x0)
n = size(A, 1); % nb de lignes
iter=0;
itermax=100;
epsilon=1e-6;
```

```
while (iter < itermax)
    u = A*x0;
    x = u / norm(u);
    if (norm(x-x0) < epsilon)
        disp(norm (x-x0));
        break;
    end
    x0=x;
    iter=iter+1;
end
```

Norme euclidienne

Rupture de la boucle
while

Execution et vérification dans scilab
[l, u]=puissance(A, x0)

```
u = x; % x de norme 1
lambda = u' * A * u;
disp(['nb iterations' num2str(iter)]);
end
```

Que donnent $\text{eig}(A)$ et $[V, D]=\text{eig}(A)$?

Annexe : fonctions graphiques évoluées

Pour $x \in [-50, 50]$, calculer $f = x \sin(x)$ puis tracer $f(x)$

Liste des fonctions MATLAB à voir : plot, xlabel, ylabel, title, grid on

Pour $\varphi \in [0, 2\pi]$ et $\theta \in [0, \pi]$ calculer $e = \sin(\theta)^4 \cos(2\varphi)$

Représenter $e(\varphi, \theta)$ sur la forme d'une surface et les isovaleurs de $e(\varphi, \theta)$ sous la forme de lignes de contour

Liste des fonctions MATLAB à voir : meshgrid, surf, contour

```
% Isovaleurs
```

```
clc
```

```
close all
```

```
[phi,tet]=meshgrid(0:0.02:2*pi,0:0.01:pi);  
e=sin(tet).^4.*cos(2*phi);
```

```
% isovaleurs fixees
```

```
v=-1.:0.2:1;
```

```
colormap([0 0 0])
```

```
[c,h]=contour(phi,tet,e,v);
```

```
set(h,'ShowText','on','TextStep',...  
    get(h,'LevelStep')*2);
```

```
figure
```

```
colormap(jet)
```

```
[c,h]=contour(phi,tet,e,v);
```

```
set(h,'ShowText','on','TextStep',...  
    get(h,'LevelStep')*2);
```

```
figure
```

```
colormap(jet)
```

```
h=surf(phi,tet,e);
```

```
shading interp
```